3 Costrutti `while`, `for` e `switch`

Questa dispensa propone esercizi sulla scrittura di algoritmi, in linguaggio C, utili alla comprensione dei costrutti `while`, `for` e `switch`.

I costrutti per costruire cicli in C sono il `while`, la variante `do...while` e il `for`.

```
inizializzazione; //opzionale

while (condizione) {
  corpo;

  incremento; //opzionale
}
```

La `condizione` è valutata prima di ogni iterazione, inclusa la prima; quindi il corpo del ciclo (e quindi anche l'eventuale istruzione di `incremento`) potrebbero non eseguire mai, nel caso in cui la condizione sia falsa dall'inizio.

```
inizializzazione; //opzionale

do {
  corpo;

  incremento; //opzionale
} while (condizione);
```

Invece, nella variante `do...while`, il corpo e l'eventuale istruzione di incremento sono eseguiti almeno una volta prima di valutare la condizione.

Il `for` è equivalente, ma ha una sintassi più compatta:

```
for (inizializzazione; condizione; incremento) {
  corpo;
}
```

Si noti che le parentesi sono necessarie solo nel caso di corpo con più di un'istruzione. Inoltre, l'espressione di inizializzazione e di incremento sono opzionali. Di fatto, un ciclo `while` può essere scritto in modo equivalente con un costrutto `for` nel seguente modo:

```
inizializzazione;

for (; condizione; ) {
  corpo;

  incremento;
}
```

Il costrutto del C che permette di scegliere tra alternative multiple (più di 2) è lo `switch`.

```
inizializzazione;

switch (variabile) {
  case valore1:
      istruzione1;
      break;
  case valore2:
      istruzione2;
      break;
  default:
      istruzione3;
      break;
}
```

Il `break` tra un case e l'altro permette che venga eseguito solo il codice relativo al case in cui si è capitati. Il `break` finale non è necessario ma consigliato (nel caso in cui si dovesse immettere nuovi case).

3.0.1 Esercizi

Esercizio 3.1

Data la seguente operazione matematica:

$$y = \sum_{i=0}^{n} i$$

Si scriva un programma per il calcolo di y. Si assuma di acquisire n come dati di input. Prima di procedere con il calcolo di y, si controlli che n sia un intero positivo.

Esercizio 3.2

Scrivere un programma che dato un numero positivo ne restituisca la radice intera

Esercizio 3.3

1. Realizzare un programma che legga da input un voto (carattere da 'A' ed 'E') e ne stampi il significato (A significa ottimo, E significa gravemente insuff.,etc.)

2. Supporre che l'utente possa inserire valori definiti con una codifica errata. Si ripeta l'acquisizione dei dati finchè non vengono inseriti valori validi.

Esercizio 3.4

1. Scrivere un programma che dato un numero reale positivo l e un intero positivo n restituisca l'area del poligono regolare con n lati di lunghezza l. Si implementi una soluzione per che gestisca i casi $n \in \{1, \ldots, 6\}$ e che sia espandibile agevolmente. Suggerimento: l'area di un poligono regolare è:

$$A = \frac{nl^2}{4\tan(\frac{\pi}{n})} \tag{3.1}$$

2. Supporre ora che l'utente possa inserire valori negativi per l e n. Si ripeta l'acquisizione dei dati finché l'utente non inserisce dei valori accettabili per i due parametri.

Esercizio 3.5

Scrivere un programma che richiede all'utente un intero positivo e ne stampa a schermo tutti i divisori.

Esercizio 3.6

Scrivere un programma che richiede all'utente un intero positivo e determina se è primo o meno. Il programma deve continuare a chiedere il numero fino a che l'utente non ne inserisce uno positivo.

Esercizio 3.7

Scrivere un programma che stampi a video la tavola pitagorica da 1-N. Dove N viene inserito dall'utente.

Esercizio 3.8

Scrivere un programma che richiede all'utente un intero positivo N e stampa a schermo i primi N numeri primi. Ad esempio: con $N = 7$ a schermo avremo 2 3 5 7 11 13 17.

Esercizio 3.9

Scrivere un programma che richiede due numeri e ne stampa il massimo comune divisore (MCD) e il minimo comune multiplo (mcm).

Esercizio 3.10

Implementare il gioco della morra cinese, noto anche come "carta-forbice-sasso". Il programma si aspetta da tastiera l'input dal giocatore 1 e, dopo, l'input dal giocatore 2. L'input potrà essere uno dei caratteri 'C', 'F' o 'S'. Il programma dovrà continuare a chiedere input ai due giocatori fino a che ognuno dei due giocatori non inserisca una giocata valida. In questo caso calcola il vincitore (giocatore 1 o 2 o pareggio), stampa a video il vincitore e termina.

Esercizio 3.11

Realizzare un programma che prenda in input una sequenza di cifre (tra 1 e 9) e calcoli la somma massima fra le sotto-sequenze di cifre non decrescenti. Il programma termini quando viene inserito lo 0.

Esempio:

2 2 4 5	3 9	3	1 5	0
13	12	3	6	

Esercizio 3.12

Dati due numeri m e n questi sono numeri amicali se la somma dei divisori di m è uguale a n, e viceversa (per esempio 220 e 284, 1184 e 1210, 2620 e 2924, 5020 e 5564, 6232 e 6368, 17296 e 18416). Ad esempio si ha:

$$220 \rightarrow 110 + 55 + 44 + 22 + 20 + 11 + 10 + 5 + 4 + 2 + 1 = 284$$
$$284 \rightarrow 142 + 71 + 4 + 2 + 1 = 220$$

Scrivere un programma che ricevuto in ingresso due numeri interi restituisce 1 se i numeri sono amicali, 0 altrimenti.

Esercizio 3.13

Scrivere un programma che richiede un numero e stampa se è palindromo o meno. Ad esempio il numero 121 è palindormo, mentre il numero 123 non lo è.

Esercizio 3.14

Si progetti in C un programma che legga un float, rappresentante un ammontare di euro; di seguito il programma deve leggere un tasso d'interesse (in percentuale), ed un numero di anni. Il programma deve stampare, in uscita, per ogni anno, come l'ammontare cresce con gli interessi. Si ricordi che l'interesse si calcola con la seguente formula:

$$C_{fin} = C_{in} * \left(1 + \frac{r}{100}\right)^N$$

Dove C_{fin} è il capitale finale, C_{in} è quello iniziale, r è l'interesse e N reppresenta il numero di anni in cui si applicano gli interessi.

Esercizio 3.15

Scrivere un programma che dato un numero reale x e un intero positivo n calcoli lo sviluppo di McLaurin del seno in x all'ordine n. Si ricorda che lo sviluppo di Taylor del seno è dato dalla sommatoria:

$$\sin x = \sum_{i=0}^{n} \frac{(-1)^i}{(2i+1)!} x^{2i+1}$$

Soluzioni

Soluzione dell'esercizio 3.1

```
#include <stdio.h>

int main() {
    int n, i = 0;
    int y = 0;

    printf("Inserisci un numero intero positivo n: ");
    scanf("%d", &n);

    // versione con ciclo while
    if (n < 0) {
        printf("Errore: n deve essere un intero positivo.\n");
    } else {
        while (i <= n) {
            y = y + i;
            i++;
        }
        printf("La somma da 0 a %d e' %d\n", n, y);
    }

    // versione con ciclo do...while
    if (n < 0) {
        printf("Errore: n deve essere un intero positivo.\n");
    } else {
        do {
            y = y + i;
            i++;
        } while (i <= n);
        printf("La somma da 0 a %d e' %d\n", n, y);
    }

    // versione con ciclo for
    if (n < 0) {
        printf("Errore: n deve essere un intero positivo.\n");
    } else {
        for (i = 0; i <= n; i++) {
            y = y + i;
        }
        printf("La somma da 0 a %d e' %d\n", n, y);
    }

    return 0;
}
```

Soluzione dell'esercizio 3.2

Abbiamo già visto in un'esercitazione precedente lo pseudocodice per questo algoritmo.

Iniziamo con la soluzione utilizzando il costrutto `while`

```
#include <stdio.h>

int main() {

    int n; //numero inserito dall'utente
    int radice; //radice intera di n

    printf("\nInserire un numero intero positivo: ");
    scanf("%d", &n);

    radice = n / 2;

    while (radice * radice > n)
        radice--;

    printf("La radice intera di %d e': %d \n", n, radice);

}
```

Alternativamente si può utilizzare anche il ciclo `for`

```
#include <stdio.h>

int main() {

    int n; //numero inserito dall'utente
    int radice; //radice intera di n
    printf("\nInserire un numero intero positivo: ");
    scanf("%d",&n);

    for(radice = n/2; radice*radice > n; radice--) {

    }

    printf("La radice intera di %d e': %d \n", n, radice);

}
```

Attenzione: In questa soluzione il ciclo non è vuoto, ma semplicemente l'unica operazione è il decremento della variabile che viene eseguito nella terza espressione del ciclo `for`

Soluzione dell'esercizio 3.3

1.
```
#include <stdio.h>
int main ()
{
    char voto;
    scanf(" %c", &voto);
    switch (voto) {
        case 'A':
        case 'a':
            printf("A = Ottimo\n");
            break;
        case 'B':
        case 'b':
            printf("B = Distinto\n");
            break;
        case 'C':
        case 'c':
            printf("C = Buono\n");
            break;
        case 'D':
        case 'd':
            printf("D = Sufficiente\n");
            break;
        case 'E':
        case 'e':
            printf("E = Gravemente insufficiente\n");
            break;
        default:
            printf("Valore non valido. Inserisci un voto tra
                A e E.\n");
            voto = 0; /* forza la ripetizione */
    }
    return 0;
}
```

2.
```
#include <stdio.h>

int main() {
    char voto;
    int isValido;
    do {
        printf("Inserisci un voto (A - E): ");
        scanf(" %c", &voto);

        switch (voto) {
            case 'A':
            case 'a':
                printf("A = Ottimo\n");
```

```
                break;
            case 'B':
            case 'b':
                printf("B = Distinto\n");
                break;
            case 'C':
            case 'c':
                printf("C = Buono\n");
                break;
            case 'D':
            case 'd':
                printf("D = Sufficiente\n");
                break;
            case 'E':
            case 'e':
                printf("E = Gravemente insufficiente\n");
                break;
            default:
                printf("Valore non valido. Inserisci un voto
                   tra A e E.\n");
                isValido = 0; /* forza la ripetizione */
        }

    } while (isValido == 0);

    return 0;
}
```

Nota: alternativamente, l'esercizio poteva essere risolto utilizzando una serie di costrutti `if` nidificati

```
#include <stdio.h>

int main() {
    char voto;
    int isValido = 0;

    while (!isValido) {
        printf("Inserisci un voto (A - E): ");
        scanf(" %c", &voto);

        if (voto == 'A' || voto == 'a') {
            printf("A = Ottimo\n");
            isValido = 1;
        } else {
            if (voto == 'B' || voto == 'b') {
                printf("B = Distinto\n");
                isValido = 1;
```

```
            } else {
                if (voto == 'C' || voto == 'c') {
                    printf("C = Buono\n");
                    isValido = 1;
                } else {
                    if (voto == 'D' || voto == 'd') {
                        printf("D = Sufficiente\n");
                        isValido = 1;
                    } else {
                        if (voto == 'E' || voto == 'e') {
                            printf("E = Gravemente
                                insufficiente\n");
                            isValido = 1;
                        } else {
                            printf("Valore non valido.
                                Inserisci un voto tra A e E.\n
                                ");
                        }
                    }
                }
            }
        }
    }

    return 0;
}
```

Soluzione dell'esercizio 3.4

1.

```
#include <stdio.h>
#include <math.h>

int main(){

    float l; //lato del poligono
    int n; //numero di lati
    float area; //area del poligono regolare con n lati
        lunghi l

    printf("Inserire la lunghezza del lato ");
    scanf("%f",&l);
    printf("Inserire il numero di lati ");
    scanf("%d",&n);

    switch (n) {
        case 1:
        case 2:
            printf("Il poligono e' degenere");
```

```
            break;
        case 3:
            area = l * l * sqrt(3) / 4;
            break;
        case 4:
            area = l * l;
            break;
        case 5:
            area = l * l * 5 / 2 * sqrt(sqrt(5)/10 + 0.25);
            break;
        case 6:
            area = l * l * sqrt(3) * 3 / 2;
            break;
        default:
            printf("Poligono non supportato");
    }

    if (n > 2 && n < 7)
        printf("L'area del poligono regolare con %d lati e' %
           f", n, area);

}
```

2.

```
#include <stdio.h>
#include <math.h>

int main(){

    float l; //lato del poligono
    int n; //numero di lati
    float area; //area del poligono regolare con n lati
       lunghi l

    do {
        printf("Inserire la lunghezza del lato ");
        scanf("%f",&l);
        printf("Inserire il numero di lati ");
        scanf("%d",&n);
        if (l <= 0 || n < 1)
            printf("Immissione scorretta. Inserire due numeri
                positivi\n");
    } while (l <= 0 || n < 1);

    switch (n) {
        case 1:
        case 2:
            printf("Il poligono e' degenere");
            break;
        case 3:
```

```
            area = l * l * sqrt(3) / 4.0;
            break;
        case 4:
            area = l * l;
            break;
        case 5:
            area = l * l * 5 / 2 * sqrt(sqrt(5)/10 + 0.25);
            break;
        case 6:
            area = l * l * sqrt(3) * 3 / 2.0;
            break;
        default:
            printf("Poligono non supportato");
    }

    if (n > 2 && n < 7)
        printf("L'area del poligono regolare con %d lati e' %
           f", n, area);

}
```

Soluzione dell'esercizio 3.5

```
#include <stdio.h>

int main() {

    int n, i;

    // ciclo di acquisizione
    do {
        printf("Inserire un intero ");
        scanf("%d" , &n);
    } while(n < 1);

    // soluzione con il ciclo while
    i = 1;
    while(i <= n / 2) {
        if(n % i == 0)
            printf(" %d " , i);

        i++;
    }
}
```

```
#include <stdio.h>

int main() {
```

```
    int n, i;

    // ciclo di acquisizione
    do {
        printf("Inserire un intero ");
        scanf("%d" , &n);
    } while(n < 1);

     // soluzione con il ciclo for
    for(i = 1; i <= n / 2; i++)
        if(n % i == 0)
            printf(" %d ", i);

}
```

Attenzione: L'`init_expr i = 1;` e la `loop_expr i++;` fanno parte della prima riga nel ciclo. Nel ciclo `while` erano prima del ciclo e all'interno rispettivamente.

Soluzione dell'esercizio 3.6

```
#include <stdio.h>

int main(){

    int n, i, primo;

    do {
        printf("Inserire un intero: ");
        scanf("%d" , &n);
    } while(n < 1);

    /*
      Variabile di flag: il suo valore cambia da 1 a 0 quando
          trovo un
      divisore. Nell'inizializzazione della variabile, assumiamo
          che
      il numero inserito sia primo.
    */
    primo = 1;

    /*
      Occorre escludere 1 e n perche' dobbiamo considerare solo i
      divisori propri. Quindi iniziamo da 2.
     */
    i = 2;
```

```
    /*
      Inutile controllare i numeri da n/2 + 1 a n: non contengono
         divisori.

      Arresto il ciclo quando la variabile di flag primo diventa
         0: è
      inutile cercare altri divisori
    */
    while (i <= n/2 && primo == 1) {
        if (n % i == 0) //se divisore trovato -> numero non primo!
            primo = 0;
        i++;
    }

    /*
      Qui, al termine del ciclo, se primo == 1 vuol dire che non
         si è
      mai verificato n % i == 0, quindi non esistono divisori
         propri
      ed n è primo.
    */

    printf("\n%d", n);

    if (primo == 0)
        printf(" non"); // il corpo dell'if è solo quest'
            istruzione
    printf(" e' primo\n");
}
```

```
#include <stdio.h>

int main() {
    int n, i, trovato;

    do {
        printf("Inserire un intero: ");
        scanf("%d" , &n);
    } while(n < 1);

    for (i = n/2; i > 1 && n % i != 0; i--);

    /*
       In alternativa, risolvendo con ciclo while:

       i = n/2;
       while (i > 1 && n % i != 0)
           i--;
     */
```

```
    if (i > 1)
        printf("Divisore %d trovato: numero non primo.", i);
    else // vale la condizione i == 1
        printf("Divisore non trovato: numero primo.");
}
```

Soluzione dell'esercizio 3.7

```
#include <stdio.h>

int main() {
    int i, j, N;

    printf("Programma per stampare la tavola pitagorica fino a N\n
       ");
    printf("Inserisci il valore di N (positivo): ");
    scanf("%d", &N);

    printf("\nTavola pitagorica (1 - %d)\n\n", N);

    /* Corpo della tavola */
    for (i = 1; i <= N; i++) {
        for (j = 1; j <= N; j++) {
            printf("%4d", i * j);
        }
        printf("\n");
    }

    return 0;
}
```

Soluzione dell'esercizio 3.8

```
#include <stdio.h>

int main(){

    int n_primi = -2;
    int n = 2;
    int i;
    int divisore;
    int flag = 1;

    // ciclo di acquisizione
    while (n_primi < 1) {
        printf("Numero positivo: ");
```

```
        scanf("%d",&n_primi);
    }

    // ciclo di ricerca degli n_primi numeri primi
    i = 1;
    while (i <= n_primi) {
        flag = 1;
        for(divisore = 2; divisore <= n / 2 && flag == 1; divisore
           ++) {
            if (n % divisore == 0) {
                flag = 0;
            }
        }

        if (flag == 1) {
            printf("%d, ",n);
            i++;
        }
        n++;
    }
}
```

Soluzione dell'esercizio 3.9

```
#include <stdio.h>

int main() {
    int n, m, MCD, mcm, i;

    do {
        printf("Inserire n = ");
        scanf("%d", &n);

        printf("Inserire m = ");
        scanf("%d", &m);

    } while (n < 1 || m < 1);

    MCD = 1;
    for (i = 1; i <= n && i <= m; i++) {
        if (n % i == 0 && m % i == 0)
            MCD = i;
    }

    /* mcm(a, b) = (a * b) / MCD(a, b) */
    mcm = (n * m) / MCD;

    printf("L'MCD di %d e %d e' %d\n", n, m, MCD);
    printf("Il m.c.m. di %d e %d e' %d\n", n, m, mcm);
```

```
    return 0;
}
```

Nota: Un metodo più efficiente per trovare MCD è l'utilizzare l'algoritmo di Euclide.

```
#include <stdio.h>

int main() {
    int n, m, a, b, resto, MCD, mcm;

    /* Acquisizione con controllo di validità */
    do {
        printf("Inserire n = ");
        scanf("%d", &n);

        printf("Inserire m = ");
        scanf("%d", &m);

    } while (n < 1 || m < 1);
    /* Si ripete finché almeno uno dei due è non positivo */

    /* Salviamo i valori originali per il calcolo dell'mcm */
    a = n;
    b = m;

    /* Algoritmo di Euclide per l'MCD */
    while (b != 0) {
        resto = a % b;
        a = b;
        b = resto;
    }

    MCD = a;
    mcm = (n * m) / MCD;

    printf("L'MCD di %d e %d e' %d\n", n, m, MCD);
    printf("Il m.c.m. di %d e %d e' %d\n", n, m, mcm);

    return 0;
}
```

Soluzione dell'esercizio 3.10

```
#include <stdio.h>

int main() {
```

```
char g1, g2;

/*
  variabile che indica il vincitore:
     - se vince == 1 allora vince g1
     - se vince == 2 allora vince g2
     - se vince == 0 allora parita'
 */
int vince = 1;

int ok1, ok2;

do {
    printf("Giocata giocatore 1 (C/F/S?): ");
    scanf(" %c", &g1);

    // fflush(stdin);

    printf("\nGiocata giocatore 2 (C/F/S?): ");
    scanf(" %c", &g2);

    // fflush(stdin);

    // variabile ausiliaria, vale 1 se la giocata del
       giocatore 1 è buona
    ok1 = (g1 == 'C' || g1 == 'F' || g1 == 'S');

    // variabile ausiliaria, vale 1 se la giocata del
       giocatore 2 è buona
    ok2 = (g2 == 'C' || g2 == 'F' || g2 == 'S');

    /*
      la condizione di permanenza nel ciclo è: "quello
         inserito da
      g1 non va bene o quello inserito da g2 non va bene"
    */
} while(!ok1 || !ok2);

/*
  per ridurre il numero di casi dell'if posso assumere che
     vinca
  sempre g1 e poi valuto le condizioni che farebbero vincere
     g2
 */

// i caratteri nel codice vanno tra apici singoli ' '
if(g1 == 'C' && g2 == 'F')
    vince = 2;
```

```
    if(g1 == 'F' && g2 == 'S')
        vince = 2;

    if(g1 == 'S' && g2 == 'C')
        vince = 2;

    // e quelle che farebbero pareggiare
    if(g1 == g2)
        vince = 0;

    // stampo il risultato
    if (vince > 0)
        printf("vince il giocatore %d\n" , vince);
    else
        printf("pari\n");

    return 0;
}
```

Soluzione dell'esercizio 3.11

```
#include <stdio.h>

int main(){
    int cifra;
    int curVal = 0, oldVal = 0, somma = 0, sommaMax = 0;

    do
    {
        printf("Inserisci una cifra (0-9): ");
        scanf("%d", &cifra);
        curVal = cifra;
        if (curVal < oldVal)
        {
            if (somma > sommaMax){
                sommaMax = somma;
            }
            somma = curVal;
        }
        else{
            somma = somma + curVal;
        }

        oldVal = curVal;
    } while (curVal != 0);

    printf("Massima somma trovata: %d\n", sommaMax);

    return 0;
```

```
}
```

Soluzione dell'esercizio 3.12

```
#include <stdio.h>

int main(){

    int n,m;
    int sum_n, sum_m;
    int i;
    printf("Primo numero: ");
    scanf("%d",&n);
    printf("Secondo numero: ");
    scanf("%d",&m);

    printf("Divisori di %d: ",n);
    sum_n = 0;
    for (i = 1; i <= n / 2; i++) {
        if (n % i == 0) {
            printf("%d,",i);
            sum_n += i;
        }
    }
    printf("\n");

    printf("Divisori di %d: ",m);
    sum_m = 0;
    for (i = 1; i <= m / 2; i++) {
        if (m % i == 0) {
            printf("%d,",i);
            sum_m += i;
        }
    }
    printf("\n");

    if (sum_n == m && sum_m == n)
        printf("I due numeri sono amicali\n");
    else
        printf("I due numeri non sono amicali\n");

}
```

Attenzione: L'algoritmo che viene presentato non è efficiente. Volendo si potrebbe minimizzare il numero delle iterazioni considerate.

Soluzione dell'esercizio 3.13

```c
#include <stdio.h>

int main(void) {
    int n, originale, invertito, cifra;

    /* Acquisizione e validazione */
    do {
        printf("Inserisci un numero intero positivo: ");
        scanf("%d", &n);
        if (n < 0) {
            printf("Il numero deve essere positivo.\n");
        }
    } while (n < 0);

    originale = n;
    invertito = 0;

    /* Calcolo del numero invertito */
    while (n > 0) {
        cifra = n % 10;      /* Prendo lultima cifra */
        invertito = invertito * 10 + cifra; /* Aggiungo la cifra
            in fondo al numero invertito */
        n = n / 10;      /* Rimuovo lultima cifra */
    }

    /* Verifica */
    if (originale == invertito) {
        printf("%d è un numero palindromo.\n", originale);
    } else {
        printf("%d non è un numero palindromo.\n", originale);
    }

    return 0;
}
```

Soluzione dell'esercizio 3.14

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    float c_in;
    float c_finale;
    float tasso, tasso_perc, tasso_tot, tasso_anni;
    int anni;
    int i, j;
```

```
    printf("Inserisci capitale iniziale: ");
    scanf("%f", &c_in);
    printf("Inserisci tasso d'interesse: ");
    scanf("%f", &tasso);
    printf("Inserisci numero di anni: ");
    scanf("%d", &anni);

    printf("Capitale iniziale: %6.2f\n", c_in);
    for (i=1; i<=anni; i++)
    {
        tasso_anni = 1;
        tasso_perc = tasso/100;
        tasso_tot = 1 + tasso_perc;

        for (j=1; j<=i; j++){
            tasso_anni=tasso_anni*tasso_tot;
        }

        c_finale = c_in * tasso_anni;
        printf("Capitale dopo %d anni: %6.2f\n", i, c_finale);
    }
    return 0;
}
```

Soluzione dell'esercizio 3.15

```
#include <stdio.h>
#include <math.h>

int main() {

    int n, i, j;
    float x, res;
    double fatt;
    printf("Programma per calcolare lo sviluppo di McLaurin di sin
       (x) all'ordine n\n");
    printf("Inserire un valore di x: ");
    scanf("%f",&x);

    printf("Inserire un valore di n: ");
    scanf("%d",&n);

    res = 0.0;
    fatt = 1.0;
    for(i = 0; i < n; i++) {
        fatt = 1;
        for(j = 1; j <= 2 * i + 1; j++)
            fatt = fatt * j;
        res = res + pow(-1.0, i) / fatt * pow(x, 2 * i + 1);
```

```
        printf("%f\n", fatt);
    }

    printf("Lo sviluppo di sin(x) e': %2.10f\n", res);
    printf("Il valore reale e': %2.10f", sin(x));
}
```

Nota: Il ricalcolo del fattoriale ad ogni iterazione è costoso ed inefficiente, possiamo ottenere una soluzione più efficiente calcolandolo in modo incrementale.

```
#include <stdio.h>
#include <math.h>

int main() {
    int n, i;
    double x, res, fatt;
    double segno;

    printf("Programma per calcolare lo sviluppo di McLaurin di sin
       (x) all'ordine n\n");
    printf("Inserire un valore di x: ");
    scanf("%lf",&x);

    printf("Inserire un valore di n: ");
    scanf("%d", &n);

    res = 0.0;
    fatt = 1.0;
    segno = 1.0;
    potenza = x;

    /* Primo termine: i = 0 -> x^(1) / 1! = x */
    for (i = 0; i < n; i++) {
        /* aggiorno il risultato */
        res += segno * potenza / fatt;

        /* preparo i valori per il prossimo termine */
        segno = -segno;  /* alterna il segno (-1)^i */

        /* aggiorna (2i+1)!  (2(i+1)+1)! */
        fatt *= (2.0 * i + 2.0) * (2.0 * i + 3.0);

        /* aggiorna potenza: x^(2i+3) = x^(2i+1) * x^2 */
        potenza *= x * x;
    }

    printf("Lo sviluppo di sin(x) e': %2.10f\n", res);
    printf("Il valore reale e': %2.10f", sin(x));
```

```
    return 0;
}
```